

VAULTLINE

Vaultline Secure File Vault

Software Engineering Project Report

Zero-knowledge encrypted file storage and cryptographic envelope sharing

PROJECT IDENTITY	DETAIL
Lead contributor	Ishjaap Singh
Registration number	1024030327
Team	Vikramaditya (1024030315); Sukhansh Mittal (1024030318)
Technology	React 18 • FastAPI • SQLAlchemy • Web Crypto API
Report date	14 September 2026
Release status	Academic release candidate; automated gates passing

Prepared for academic evaluation and technical defence

Executive Summary

Vaultline is a full-stack secure file vault whose central design decision is that plaintext and usable decryption keys remain inside the user's browser. The server authenticates users, stores opaque ciphertext, records ownership and share grants, and transports wrapped keys; it is not trusted with file content, filenames, MIME types, master passwords, unwrapped private keys, or file data-encryption keys.

The finished system combines a React 18 and Vite client, browser-native Web Crypto primitives, a FastAPI REST service, SQLAlchemy persistence, SQLite for zero-setup evaluation, optional PostgreSQL for production-style deployment, Argon2id verification of client-derived authentication proofs, and signed JWT access tokens. The website also contains an interactive guided demonstration at /demo, so evaluation is not dependent on terminal choreography.

QUALITY SIGNAL	CURRENT RESULT
Frontend lint	PASS
Frontend unit/boundary tests	3 passed in 2 test files
Production frontend build	PASS; 148 modules transformed
Backend integration/security tests	4 passed
Node dependency audit	0 vulnerabilities
Repository state at inspection	Clean working tree

Key outcome

The previously broken second-login flow was repaired by persisting the exact client-generated salt and fetching it before derivation. The multipart upload failure was repaired by preventing Axios from imposing a JSON content type on FormData. These fixes are protected by automated regression tests and documented as engineering incidents rather than hidden implementation details.

Contents

1. Project Definition and Scope
2. Requirements Specification
3. Technology Stack
4. Architecture and Trust Boundaries
5. UML and Data Diagrams
6. Implementation Details
7. Authentication and Cryptographic Design
8. API and Persistence Design
9. Security and Threat Model
10. Testing and Verification
11. User Experience and In-Website Demonstration
12. Engineering Journal and Contribution Record
13. Deployment and Operations
14. Known Limitations and Roadmap
15. Conclusion
16. Appendices

1 Project Definition and Scope

1.1 Problem statement

Conventional file-storage services can often inspect content or recover encryption keys. Vaultline explores an alternative: a browser-enforced zero-knowledge boundary where the backend remains useful for identity, authorization, persistence, and sharing without learning plaintext.

1.2 Objectives

- Encrypt file bytes and sensitive metadata before network transmission.
- Generate one independent AES data-encryption key per file.
- Allow multi-user sharing by wrapping the same DEK for each recipient.
- Keep the user private key encrypted at rest and unwrapped only in volatile session memory.
- Provide persistent, repeatable authentication and authorization.
- Deliver a refined editorial interface and a self-contained in-website product demonstration.
- Provide reproducible tests, deployment instructions, architecture records, and weekly journals.

1.3 Scope boundaries

The release supports registration, login, logout, authenticated file upload/list/detail/download/delete, public-key lookup, grant listing, sharing, revocation, files-shared-with-me, health checks, API documentation, and the guided demo. It does not claim hard cryptographic revocation of previously downloaded copies, multi-gigabyte streaming encryption, or protection against a malicious public-key substitution service.

2 Requirements Specification

ID	FUNCTIONAL REQUIREMENT	EVIDENCE
FR-01	Register with client-generated key material	POST /api/auth/register
FR-02	Repeat login using the persisted original salt	fetch-salt + login regression test
FR-03	Encrypt content and metadata locally	src/crypto/fileCrypto.js
FR-04	Upload only ciphertext and wrapped material	multipart /api/files/upload
FR-05	List, inspect, decrypt and download authorized files	file routes + client service
FR-06	Share by recipient public-key envelope	/api/sharing/share
FR-07	Revoke future server access	/api/sharing/revoke
FR-08	Run guided tour in the website	React route /demo

QUALITY ATTRIBUTE	TARGET / DESIGN RESPONSE
Confidentiality	AES-256-GCM; no plaintext server storage
Integrity	GCM authentication tag rejects modification
Authentication	PBKDF2/HKDF client derivation + Argon2id server hash + JWT
Usability	Warm editorial UI; dashboard and guided demo
Portability	SQLite default; PostgreSQL override
Maintainability	Layered UI/context/crypto/service/API/ORM structure
Auditability	Tests, changelog, canonical docs, individual and combined journals

3 Technology Stack

LAYER	TECHNOLOGY	RESPONSIBILITY
Presentation	React 18.3, React Router 7, CSS	Pages, layouts, components, navigation and design system
Build and QA	Vite 8, Vitest 5, ESLint 10	Development server, bundling, tests and lint
Transport	Axios 1.7	Bearer tokens, JSON calls, multipart and binary responses
Cryptography	Web Crypto API	PBKDF2, HKDF, AES-GCM, RSA-OAEP and secure randomness
API	FastAPI 0.141, Uvicorn	Validation, routing, middleware and OpenAPI
Persistence	SQLAlchemy 2, Alembic	ORM mapping and migrations
Database	SQLite / PostgreSQL	Identity, metadata and share-grant records
Server security	Passlib Argon2, python-jose	Proof hashing and JWT signing

3.1 Repository organization

```

src/                React application, crypto, state and services
backend-integration/ FastAPI service, ORM, schemas, tests and migrations
docs/              Architecture, security, API, testing and demo guides
journals/          Individual logs and combined weekly retrospective
scripts/           Reproducible changelog and report generation

```

4 Architecture and Trust Boundaries

Vaultline separates a trusted browser boundary from an untrusted transport and server boundary. Only the browser handles plaintext, the master password, the derived wrap key, the unwrapped RSA private key, and unwrapped file DEKs. The API receives an authentication proof, public key, encrypted private key, opaque ciphertext, encrypted metadata, and recipient-specific wrapped DEKs.

UML Component Diagram

Runtime modules and dependency direction

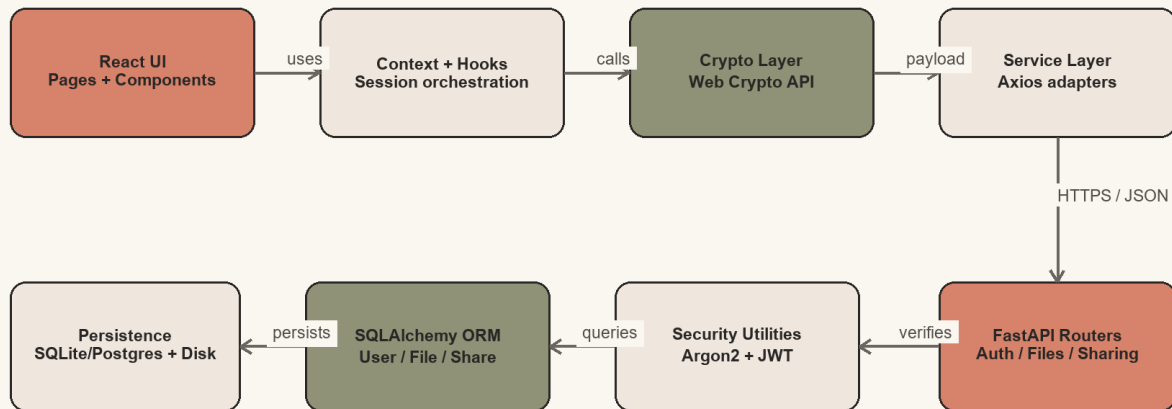


Figure 1. Component-level architecture and dependency direction.

4.1 Layer responsibilities

LAYER	KEY MODULES	RULE
UI	pages, layouts, components	No direct persistence or cryptographic shortcuts
State/orchestration	contexts and hooks	Owns session state and workflow sequencing
Crypto	src/crypto	All plaintext transformation occurs here in-browser
Services	src/services and src/api	Maps domain payloads to REST contracts
API	FastAPI routers and schemas	Authenticates, authorizes and validates
Persistence	models, database, uploads	Stores records and opaque encrypted bytes

5 UML and Data Diagrams

UML Use Case Diagram

Actors and externally visible Vaultline capabilities

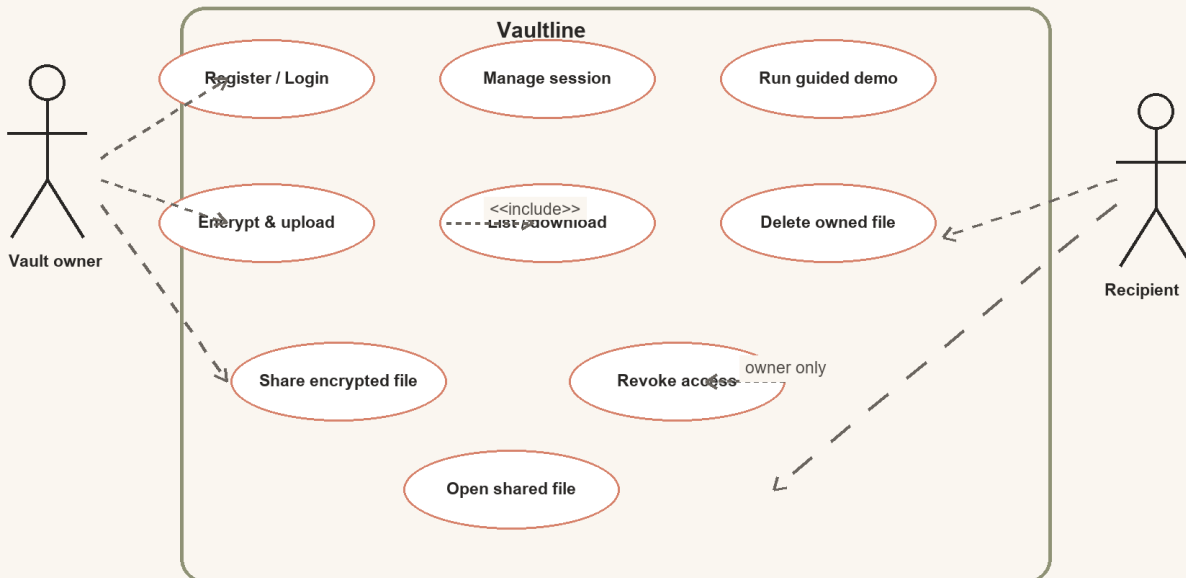


Figure 2. UML use-case diagram covering owner, recipient, and evaluation workflows.

UML Class Diagram

Core backend entities and frontend security/session classes

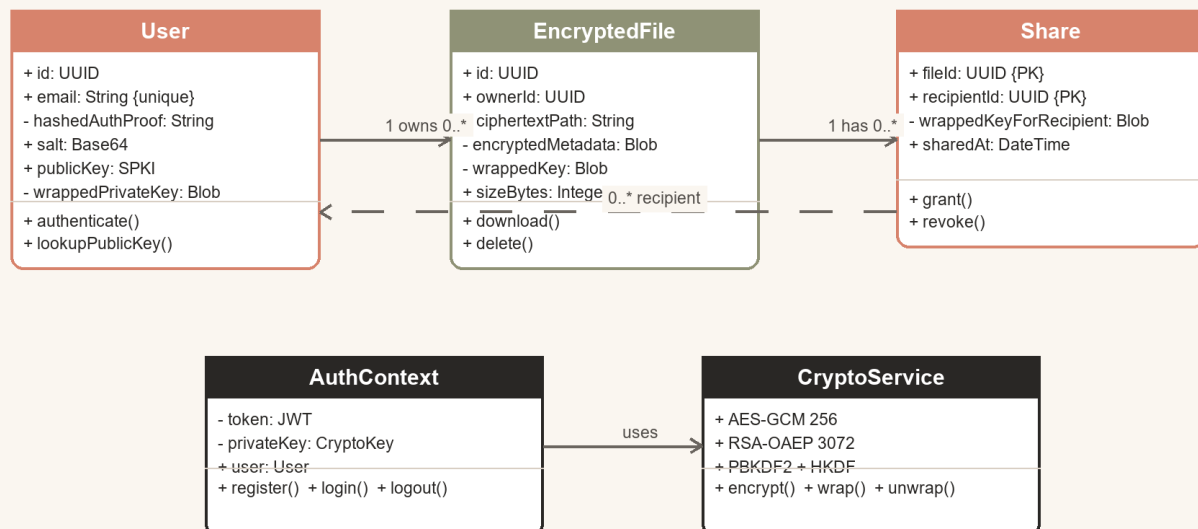


Figure 3. UML class diagram for persistent entities and core client services.

UML Sequence — Registration

UML sequence diagram — browser/server trust boundary preserved

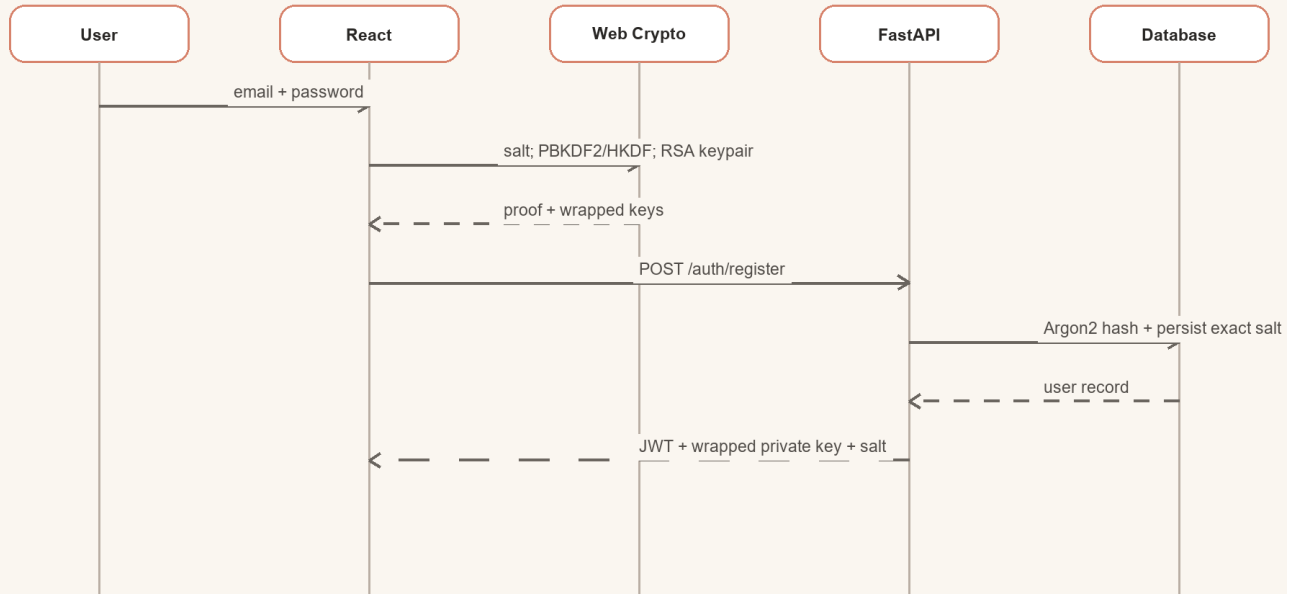


Figure 4. Registration sequence: client derivation, key generation, proof hashing, persistence and token issue.

UML Sequence — Repeat Login

UML sequence diagram — browser/server trust boundary preserved

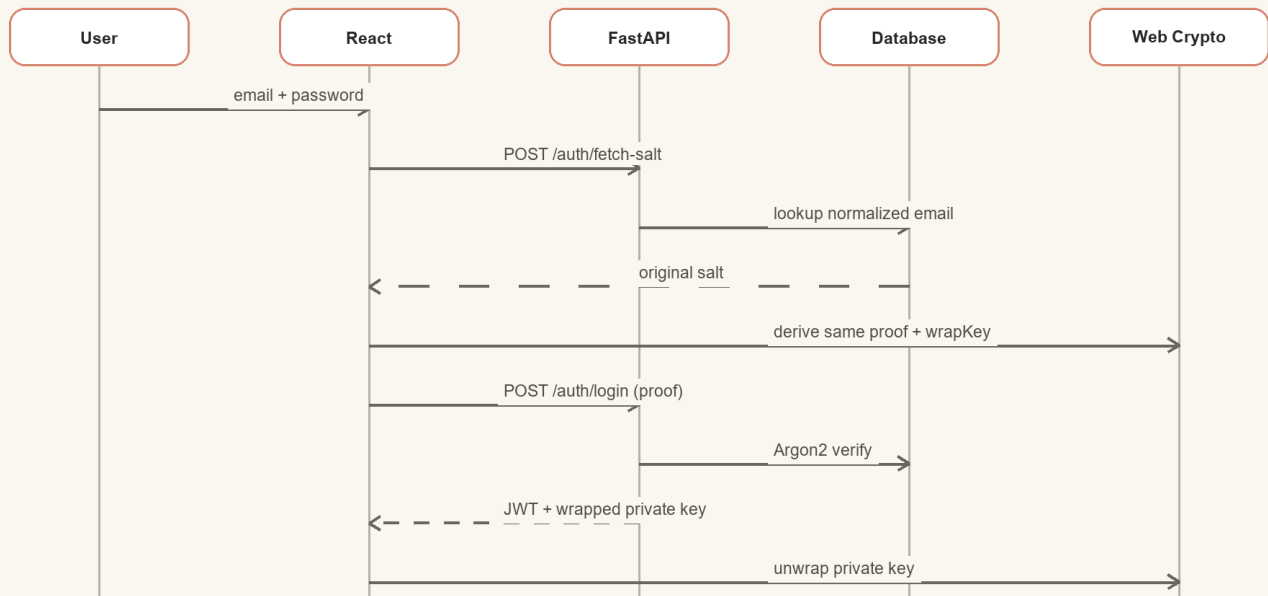


Figure 5. Repeat-login sequence: persisted salt retrieval restores deterministic derivation.

UML Sequence — Secure Sharing

UML sequence diagram — browser/server trust boundary preserved

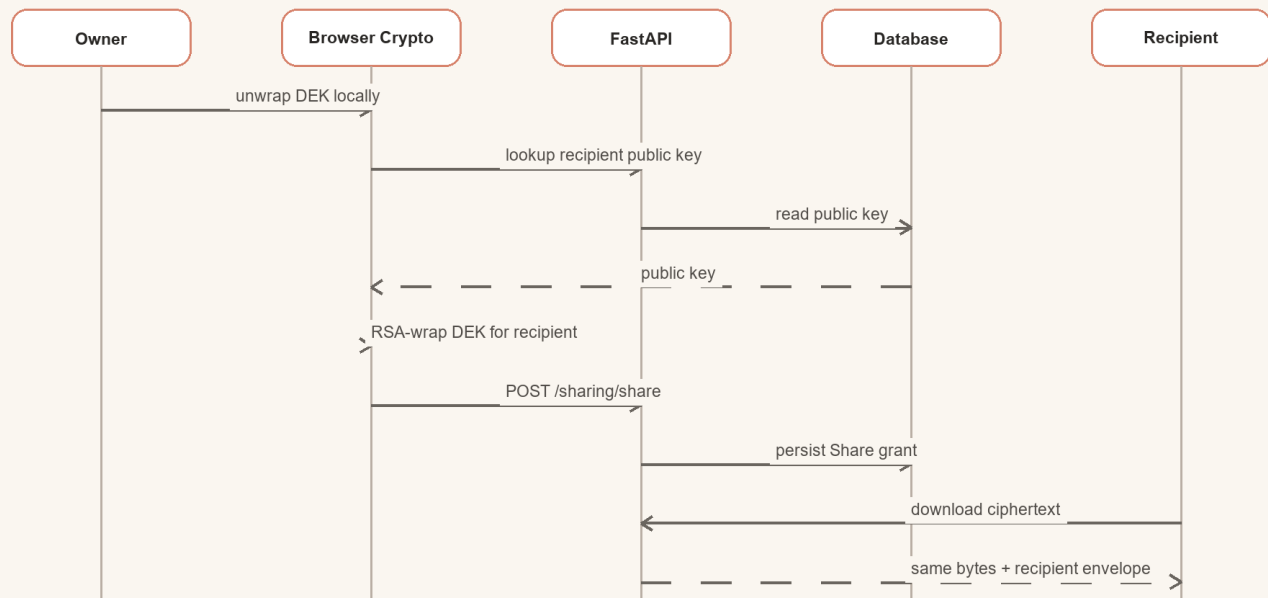


Figure 6. Secure-sharing sequence: the ciphertext is not re-uploaded; only a new recipient envelope is stored.

UML Activity — Encrypted File Lifecycle

UML activity diagram — control flow, decisions, and terminal states

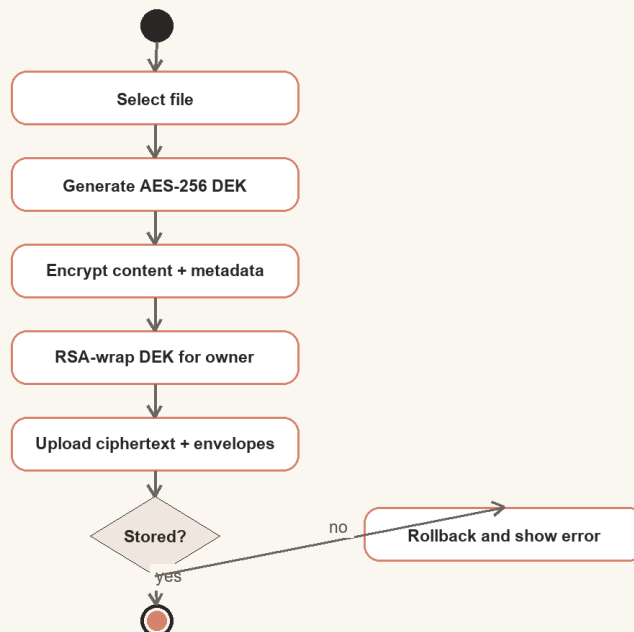


Figure 7. Activity diagram for the encrypted-file lifecycle, including rollback on storage failure.

UML Activity — Repeat Login and Recovery

UML activity diagram — control flow, decisions, and terminal states

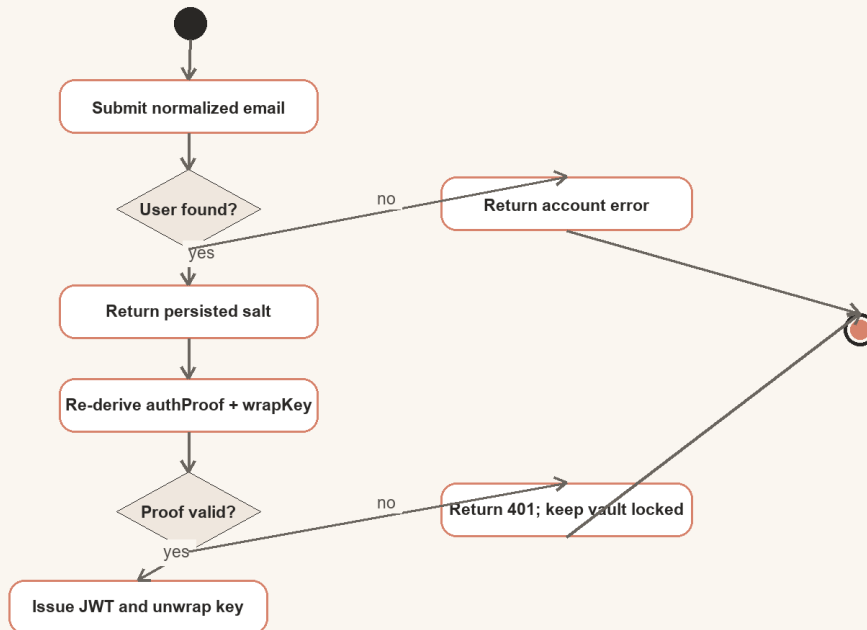


Figure 8. Activity diagram for repeat login, proof verification, rejection, and vault unlocking.

UML State Machine

Encrypted file and share-grant lifecycle

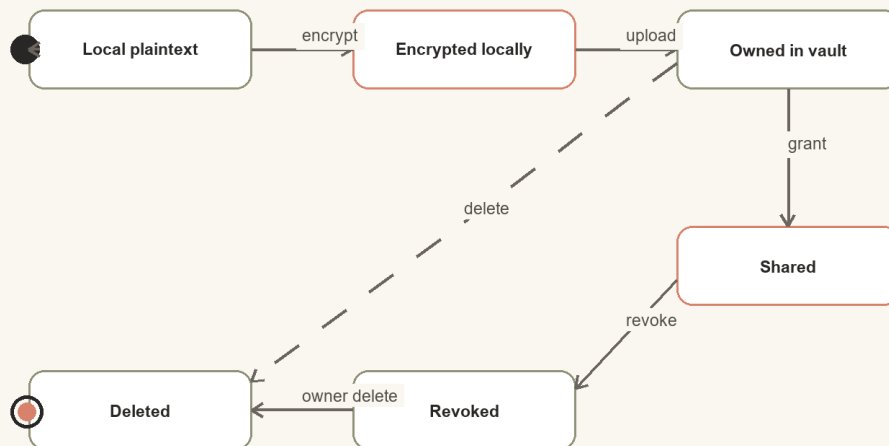


Figure 9. State machine for file and share-grant states.

UML Deployment Diagram

Development topology and production substitution points

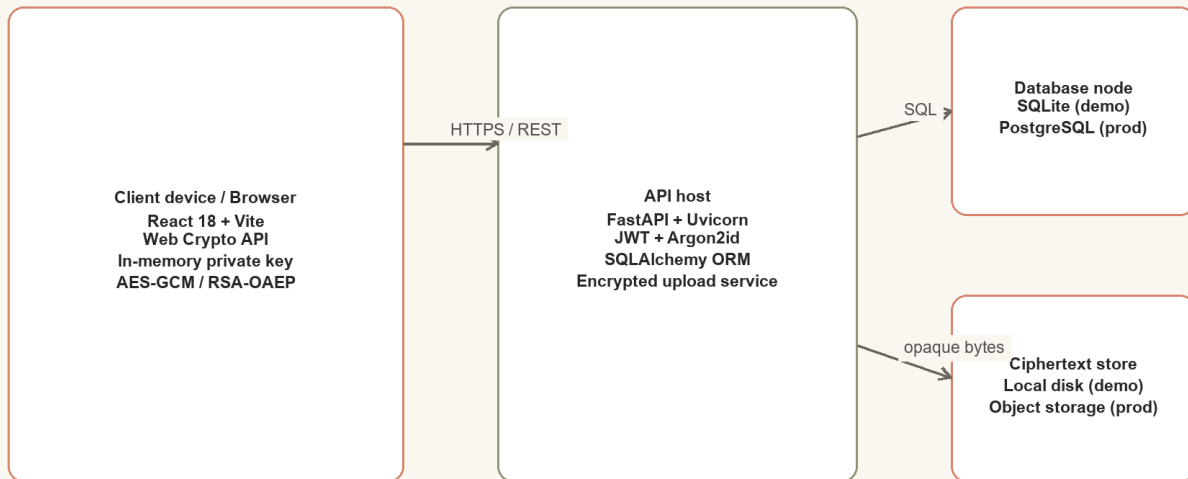


Figure 10. Deployment diagram for classroom and production profiles.

Entity Relationship Diagram

Relational persistence model and cardinalities

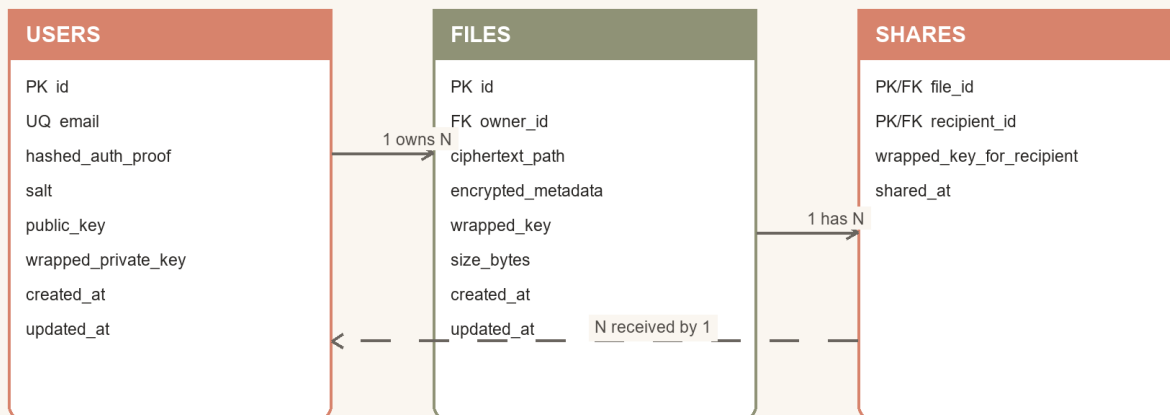


Figure 11. Entity-relationship diagram with ownership and share cardinalities.

6 Implementation Details

6.1 Frontend

App.jsx defines public landing and demo routes, authentication routes, protected application views, and shared toast/modal providers. AuthContext holds the user, JWT, and unwrapped CryptoKey in memory. useFiles coordinates encryption, upload, download, sharing and UI state. Service adapters isolate HTTP behavior, while the page and component hierarchy remains focused on interaction and presentation.

6.2 Backend

FastAPI mounts authentication, file and sharing routers under /api, exposes root and health endpoints, initializes the database during application lifespan, measures request time, configures CORS, and normalizes validation and database errors. SQLAlchemy models represent users, files and composite-key share grants. Ciphertext is stored separately on disk, with its path referenced from the database.

6.3 Fault handling

- Upload size is checked against the configured 100 MB limit.
- If disk writing fails after the database row is created, the row is rolled back.
- Owner-only operations return 403 for recipients.
- Unauthorized downloads are concealed behind not-found behavior.
- Duplicate accounts and shares return conflict responses.
- Production mode refuses the shared development signing key or a secret shorter than 32 characters.

7 Authentication and Cryptographic Design

7.1 Registration

17. Generate a random 16-byte salt in the browser.
18. Run PBKDF2-HMAC-SHA-256 for 600,000 iterations to derive 256 bits of master material.
19. Use HKDF-SHA-256 domain separation with vaultline-auth-v1 and vaultline-wrap-v1.
20. Generate a 3072-bit RSA-OAEP key pair with SHA-256.
21. Encrypt the PKCS#8 private key with the non-extractable AES-GCM wrap key.
22. Send only the auth proof, public key, wrapped private key, email and salt.
23. Store Argon2id(authProof), not the master password or raw proof.

7.2 Repeat-login defect and repair

Root cause: the old flow regenerated a new salt during later login attempts. A password KDF is deterministic only for the same password and salt, so the new salt produced a different authentication proof and a different private-key wrapping key. Authentication and private-key recovery therefore failed by design.

Repair: registration persists the exact client salt. Login first performs /auth/fetch-salt, derives with that original salt, submits the reproduced proof, receives the wrapped private key and JWT, and unwraps the private key locally. Email normalization makes case variants resolve to the same account. The backend test performs registration, salt retrieval, login, authenticated /me, a second uppercase-email login, and wrong-proof rejection.

7.3 File envelope encryption

ITEM	ALGORITHM / FORMAT	LOCATION
Per-file DEK	AES-GCM, 256-bit key	Generated in browser
Content nonce	Random 96-bit IV, packed with ciphertext	Browser-generated
Metadata	JSON encrypted independently with distinct AES-GCM IV	Browser-generated
User key pair	RSA-OAEP 3072, SHA-256	Generated in browser
Owner/recipient envelope	RSA-wrapped raw DEK	Browser-generated
Private-key protection	AES-GCM under password-derived wrap key	Browser-generated

8 API and Persistence Design

METHOD	ENDPOINT	AUTH	PURPOSE
POST	/api/auth/register	No	Create account and key record
POST	/api/auth/fetch-salt	No	Retrieve original KDF salt
POST	/api/auth/login	No	Verify proof and issue JWT
POST	/api/auth/logout	Yes	Acknowledge client-side logout
GET	/api/auth/me	Yes	Validate token/current user
POST	/api/auth/lookup-public-key	Yes	Resolve sharing recipient key
POST	/api/files/upload	Yes	Store multipart ciphertext
GET	/api/files	Yes	List owned and shared files
GET	/api/files/{id}	Yes	Return authorized encrypted metadata
GET	/api/files/{id}/download	Yes	Stream authorized ciphertext and headers
DELETE	/api/files/{id}	Owner	Delete row, grants and ciphertext
POST	/api/sharing/share	Owner	Create recipient envelope grant
GET	/api/sharing/file/{id}	Owner	List recipients
DELETE	/api/sharing/revoke	Owner	Remove future server access
GET	/api/sharing/shared-with-me	Yes	List received grants

8.1 Relational model

User owns zero or more File rows. A File owns zero or more Share rows. Share uses the composite key (file_id, recipient_id), preventing duplicate grants. Cascades remove owned files and dependent grants with their parent. Sensitive metadata and key envelopes are stored as opaque text; ciphertext bytes are stored outside the relational database.

9 Security and Threat Model

STRIDE THREAT	CONTROL	RESIDUAL RISK
Spoofing	Argon2id proof verification and signed expiring JWT	Token theft remains possible on a compromised client
Tampering	AES-GCM authenticated encryption	Availability attacks and record deletion remain possible
Repudiation	Ownership, share and timestamps	No immutable audit ledger yet
Information disclosure	Client-only plaintext; metadata encryption; wrapped keys	File size and timing metadata remain visible
Denial of service	Upload limit and validation	No distributed rate limiting in academic build
Elevation of privilege	Owner/recipient checks on every protected route	Server compromise can disrupt availability and key lookup

9.1 Explicit invariants

- The master password is never sent to the server.
- The auth proof is never used as an encryption key.
- The wrap key and unwrapped private key stay in volatile browser state.
- Each file has a fresh DEK and independent IVs for content and metadata.
- Sharing adds a wrapped DEK; it does not duplicate or decrypt the file.
- Revocation removes future API authorization but cannot erase a recipient's prior local copy.

9.2 Production hardening

- Set ENVIRONMENT=production and a unique 32+ character SECRET_KEY.
- Use HTTPS, secure reverse-proxy headers and restricted CORS origins.
- Move ciphertext to hardened object storage and PostgreSQL to managed persistence.
- Add rate limiting, structured redacted logs, backup/restore testing and secret rotation.
- Add key pinning or a signed transparency log to mitigate public-key substitution.

10 Testing and Verification

Verification was rerun on 14 September 2026 against the inspected repository. A first sandboxed frontend attempt was blocked by Windows EPERM when Vite tried to spawn its helper; the identical command passed outside that restricted execution sandbox. This is an execution-environment constraint, not a source failure.

GATE	COMMAND	RESULT
Lint + frontend tests + production build	npm run verify	PASS
Vitest	vitest run	2 files, 3 tests passed
Vite production compilation	vite build	148 modules; build passed
Backend integration/security	pytest tests -q	4 passed
Dependency vulnerability scan	npm audit --audit-level=high	0 vulnerabilities
Git working tree	git status --short	Clean at inspection

10.1 Automated coverage

TEST	BEHAVIOR PROTECTED
KDF determinism	Same password and salt reproduce the same authentication proof
Private-key recovery	RSA private key survives AES-GCM wrap/unwrap
Multipart boundary	FormData is not mutated to application/json
Repeat login	Persisted salt works across successive logins and email case variation
JWT/auth	A real JWT authorizes /auth/me; wrong proof receives 401
File lifecycle	Upload, list, binary download, headers and owner deletion
Sharing controls	Pre-share denial, grant, shared-with-me, recipient denial, revoke and post-revoke denial
Production settings	Default or short JWT secrets are rejected

10.2 Warnings

The backend suite emits dependency deprecation warnings from Starlette/httpx compatibility, anyio aliases, and Passlib's Argon2 version inspection. These warnings do not fail tests, but dependency upgrades should be scheduled before long-term production maintenance.

11 User Experience and In-Website Demonstration

The visual system replaces generic AI-dashboard styling with a warm, restrained editorial language: ivory surfaces, charcoal typography, muted coral actions, sage secondary accents, generous whitespace, serif display type, clear hierarchy, and deliberate motion. The protected vault remains task-oriented while the public landing page communicates the security model in plain language.

11.1 Guided demo

The /demo route is part of the React website and requires no terminal-only walkthrough. It presents the cryptographic lifecycle as an evaluator-friendly narrative: derive and separate keys, encrypt bytes and metadata, inspect transport-safe payloads, share through a recipient envelope, open and verify locally, and audit security guarantees.

DEMO CHAPTER	WHAT THE EVALUATOR SEES
Overview / derivation	Trusted browser boundary and password-derived separation
Encrypt	AES-GCM transformation with independent nonces
Journey	Only opaque payloads cross the network
Share	Recipient lookup and RSA re-wrapping without file duplication
Open / tamper	Local unwrap, authenticated decrypt and tamper rejection
Proof	Algorithm parameters, invariants and threat mitigations

12 Engineering Journal and Contribution Record

CONTRIBUTOR	REGISTRATION	PRIMARY CONTRIBUTION
Ishjaap Singh	1024030327	Full-stack integration, repeat-login repair, persistent auth/JWT, multipart repair, API alignment, UI redesign, in-site demo, automated tests and canonical documentation
Vikramaditya	1024030315	React component architecture, browser cryptography, encrypted metadata, envelope workflow and mock service boundary
Sukhansh Mittal	1024030318	FastAPI/SQLAlchemy foundation, Alembic setup, database troubleshooting and Python dependency triage

12.1 Week-by-week journal

WEEK	ISSUE / OBJECTIVE	RESOLUTION
1	Define trust boundary and frontend scaffold	Separated UI, state, crypto and services; chose per-file envelope encryption
2	Backend scaffold and Python 3.14 compatibility	FastAPI/SQLAlchemy/Alembic setup; Argon2 and missing validation dependencies resolved
3	Second login failed	Persisted exact salt; added fetch-salt; normalized email; protected with regression test
4	Connect encrypted file and sharing lifecycle	Implemented real service adapters, ownership checks and recipient envelopes
5	Interface appeared generic and AI-like	Introduced warm editorial Apple-inspired design language and clearer hierarchy
6	Demo existed only as technical workflow	Built interactive /demo inside the website
7	Upload boundary and release confidence	Removed global JSON header, protected FormData, added tests, production guards and canonical docs

12.2 Engineering decisions

- SQLite became the default evaluator profile to eliminate external database friction; PostgreSQL remains configurable.
- Session key material is deliberately not persisted. Refreshing requires re-authentication, preventing a plaintext-equivalent private key from resting on disk.
- Sharing uses one ciphertext and multiple small key envelopes, reducing bandwidth and server-side cryptographic trust.
- Known limitations are documented explicitly to support honest technical defence.

13 Deployment and Operations

13.1 Local evaluator runbook

```
Terminal 1
cd backend-integration
.\.venv\Scripts\python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

```
Terminal 2
npm install
npm run dev
```

SURFACE	URL
Website	http://localhost:5173
In-site guided demo	http://localhost:5173/demo
Backend API	http://127.0.0.1:8000
Interactive OpenAPI	http://127.0.0.1:8000/docs
Health check	http://127.0.0.1:8000/health

13.2 Configuration

VARIABLE	DEVELOPMENT DEFAULT	PRODUCTION ACTION
DATABASE_URL	SQLite vaultline.db	Set PostgreSQL DSN
SECRET_KEY	Shared development value	Replace with unique high-entropy secret
ENVIRONMENT	development	Set production
UPLOAD_DIR	backend-integration/uploads	Use protected durable storage
MAX_FILE_SIZE	100 MB	Tune with proxy and storage limits
CORS_ORIGINS	localhost origins	Restrict to deployed frontend

14 Known Limitations and Roadmap

LIMITATION	WHY IT MATTERS	ROADMAP
Whole-file memory buffering	Large files can exhaust browser/API memory	Chunked streaming AEAD with unique nonces
Soft revocation	Previously downloaded copies remain readable	Rotate DEK and re-encrypt for remaining users
Server-managed public keys	Malicious server could substitute a key	Key pinning, signed directory or transparency log
JWT logout acknowledgement	JWT remains valid until expiry if copied	Short-lived access tokens and refresh-token rotation
Local ciphertext disk	Single-host durability and scaling	Encrypted object storage plus lifecycle policy
Limited observability	Operational incidents may be hard to	Structured redacted audit events and

	investigate	metrics
Academic test depth	No browser E2E or load suite yet	Playwright flows, property tests, fuzzing and load tests

15 Conclusion

Vaultline is a viable, presentable academic full-stack security project with a defensible architecture and an honest threat model. Its strongest contribution is not a single cipher call; it is the end-to-end preservation of a browser-side plaintext boundary across authentication, storage, download and sharing. The critical repeat-login and multipart failures were traced to precise protocol causes, repaired at the correct layers, and converted into regression tests.

The system is ready for classroom demonstration and repository submission in its current academic-release-candidate scope. Production deployment would require the hardening and scalability items stated in this report, but those limitations do not invalidate the implemented zero-knowledge workflow.

Appendix A Evidence Map

CLAIM	PRIMARY REPOSITORY EVIDENCE
Architecture and diagrams	docs/ARCHITECTURE.md
Cryptographic model and STRIDE	docs/SECURITY.md
REST contracts	docs/API_REFERENCE.md
Verification commands and matrix	docs/TESTING.md
Presentation script	docs/DEMO_GUIDE.md
Deployment profiles	DEPLOYMENT_GUIDE.md
Release readiness	STATUS.md
Incident history	CHANGELOG.md
Team retrospective	journals/COMBINED_TEAM_JOURNAL.md
Ishjaap Singh record	journals/1024030327-Ishjaap-Singh/week1-2.md
Frontend cryptography	src/crypto/*.js
Backend identity and authorization	backend-integration/app/routers/*.py
Regression tests	src/**/*.test.js; backend-integration/tests/test_api.py

Appendix B Glossary

TERM	DEFINITION
AEAD	Authenticated encryption with associated data; provides confidentiality and integrity
AES-GCM	Symmetric authenticated cipher used for file and metadata encryption
DEK	Per-file data-encryption key
Envelope encryption	Encrypting data with a DEK and wrapping that DEK for authorized

	identities
HKDF	Key-expansion function used for domain-separated subkeys
JWT	Signed access token used for API authorization
PBKDF2	Password-based derivation function run for 600,000 iterations
RSA-OAEP	Asymmetric wrapping scheme used for owner and recipient DEK envelopes
Zero knowledge	Here: the storage service lacks plaintext and usable decryption keys

Appendix C Document References

- Vaultline source repository and canonical documentation, inspected 14 September 2026.
- NIST SP 800-38D, Galois/Counter Mode for authenticated encryption.
- NIST SP 800-132, password-based key derivation guidance.
- RFC 5869, HMAC-based Extract-and-Expand Key Derivation Function.
- RFC 7519, JSON Web Token.
- Web Cryptography API, W3C Recommendation.